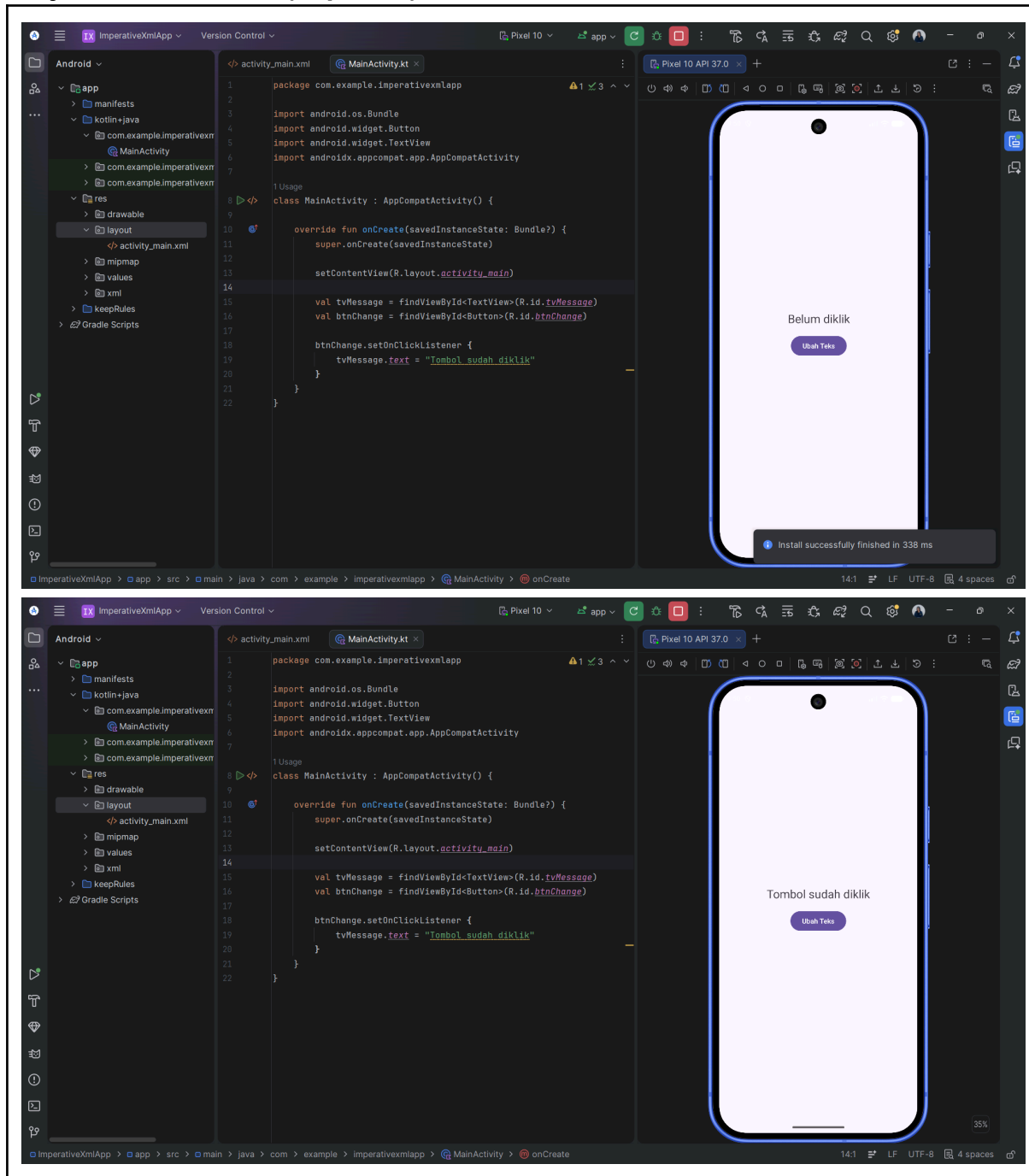


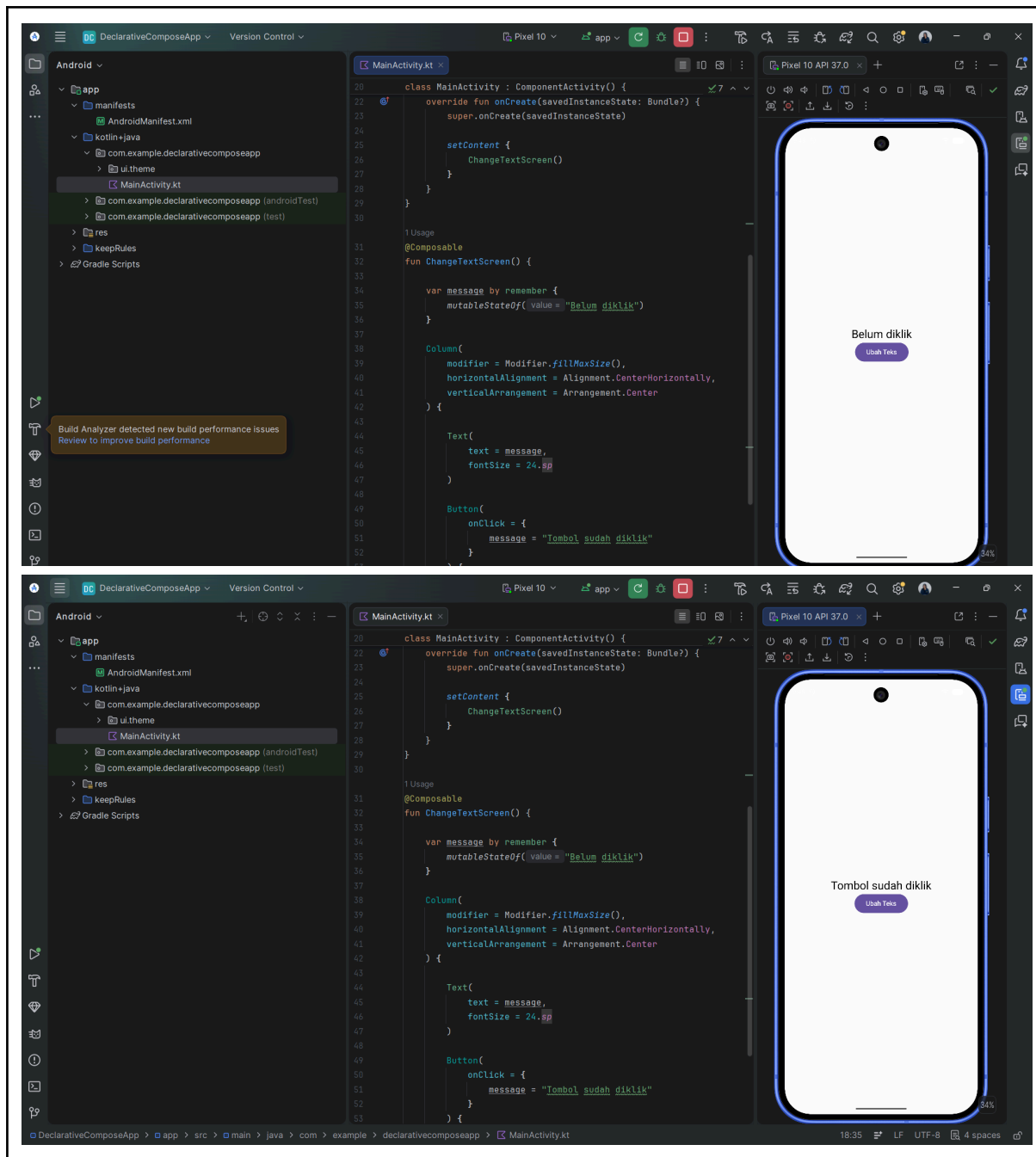
Nama : Intan Nur Azizah
NIM : 215150600111005
Kelas : PAPB - PTI A

Percobaan 1 (Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks)

Project A - XML/View (Imperatif)



Project B - Jetpack Compose (Deklaratif)



Aktivitas Percobaan

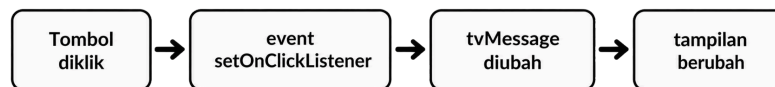
1. Screenshot sebelum dan sesudah tombol klik telah dilampirkan pada percobaan masing-masing project diatas
2. Objek yang diubah secara langsung adalah TextView tvMessage. Ketika tombol diklik, properti text dari tvMessage langsung diubah.

```
tvMessage.text = "Tombol sudah diklik"
```

3. Nilai yang diubah adalah state message. Ketika tombol diklik, nilai message berubah dari "Belum diklik" menjadi "Tombol sudah diklik".

```
message = "Tombol sudah diklik"
```

4. Karena Jetpack Compose menggunakan pendekatan deklaratif. Pada percobaan tidak mengubah komponen Text secara langsung, tetapi mengubah state yang menjadi sumber data UI. Ketika state berubah, Compose akan memperbarui bagian UI yang bergantung pada state tersebut melalui recomposition.
5. Pada XML/View, saat tombol diklik, event `setOnClickListener` dijalankan. Program kemudian mengakses objek `TextView` yaitu `tvMessage`, lalu mengubah properti text secara langsung sehingga tulisan pada layar berubah.



6. Pada Compose, saat tombol diklik, event `onClick` mengubah nilai state `message`. Karena `message` merupakan state yang dapat diamati, Compose melakukan recomposition pada UI yang menggunakan nilai tersebut. Setelah itu, Text menampilkan nilai `message` yang baru.

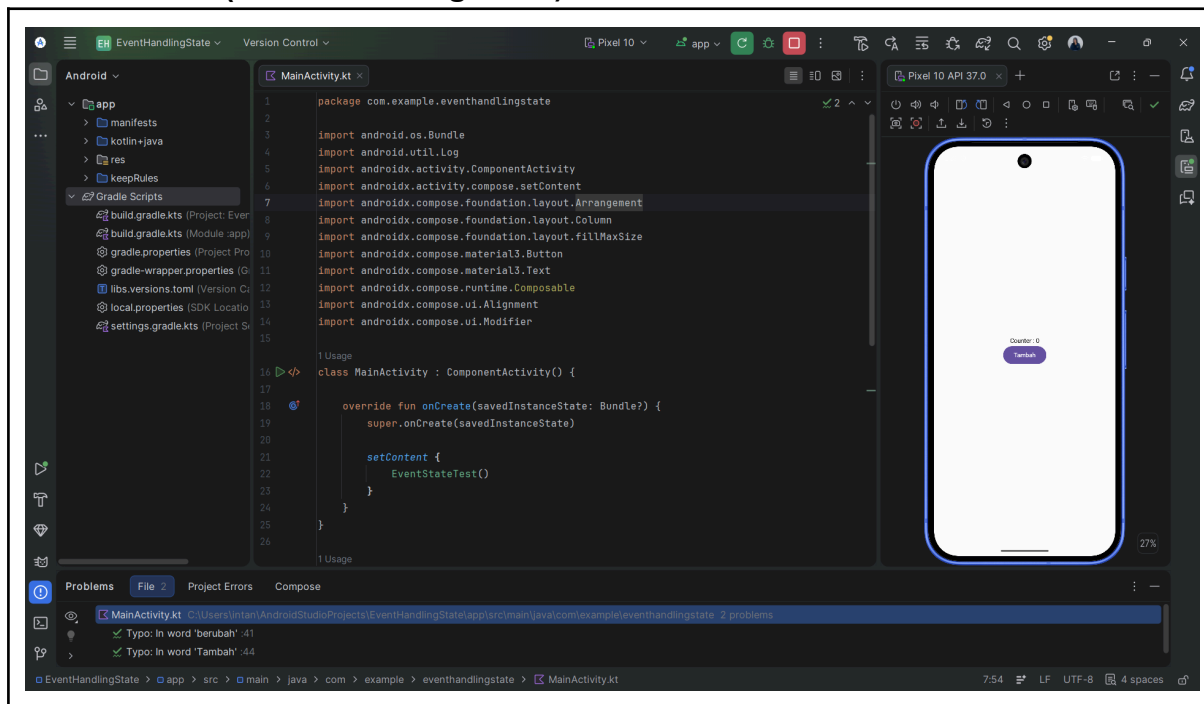


7. Jika `message` hanya menggunakan variabel biasa, perubahan nilainya tidak otomatis memperbarui tampilan. Ini karena variabel biasa bukan state yang dapat diamati oleh Compose, sehingga ketika nilainya berubah Compose tidak mendapat pemberitahuan untuk melakukan recomposition.

```
var message by remember {  
    mutableStateOf("Belum diklik")  
}
```

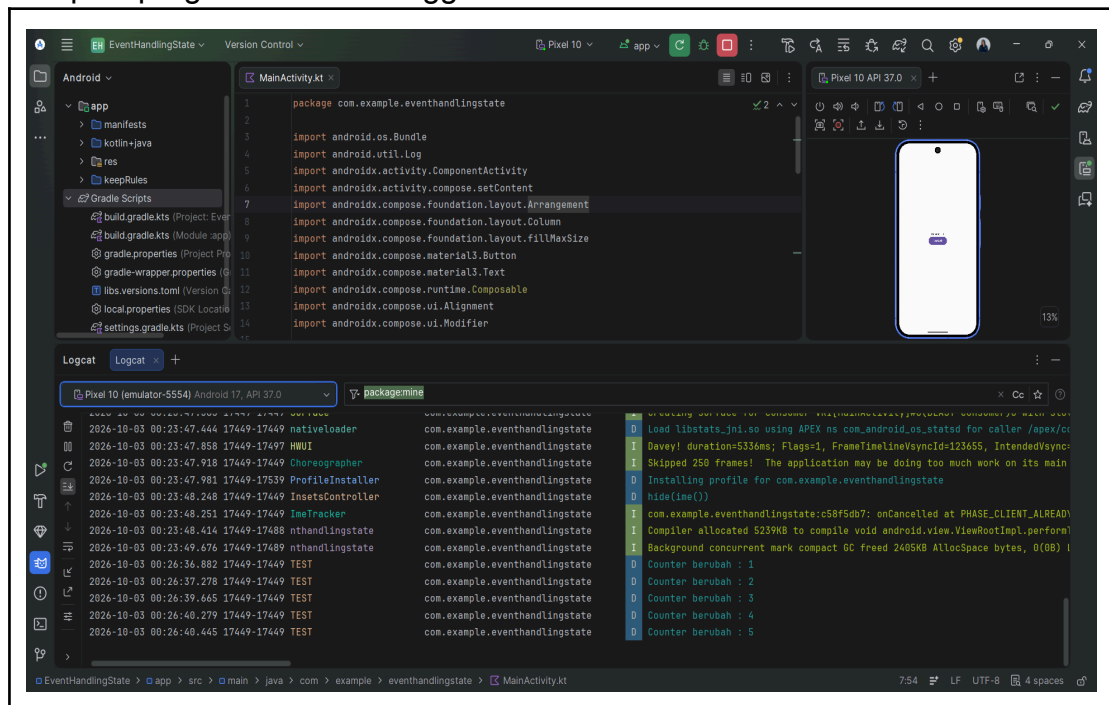
8. Menurut saya, pada pendekatan imperatif programmer harus menentukan komponen UI yang ingin diubah lalu mengubahnya secara langsung. Sedangkan pada pendekatan deklaratif, programmer cukup mengubah state, kemudian sistem akan memperbarui tampilan sesuai dengan state tersebut secara otomatis. Jadi, imperatif lebih berfokus pada cara mengubah UI, sedangkan deklaratif lebih berfokus pada bagaimana UI seharusnya tampil berdasarkan state saat ini.

Percobaan 2 (Event Handling State)



Aktivitas Percobaan

1. Setelah program dijalankan dan tombol “Tambah” diklik beberapa kali, ambil screenshot tampilan aplikasi dan LogCat sebagai bukti percobaan. Pada tahap ini program masih menggunakan var count = 0



2. Pada LogCat muncul pesan bahwa nilai counter berubah, pesan tersebut muncul karena setiap di click menjalankan

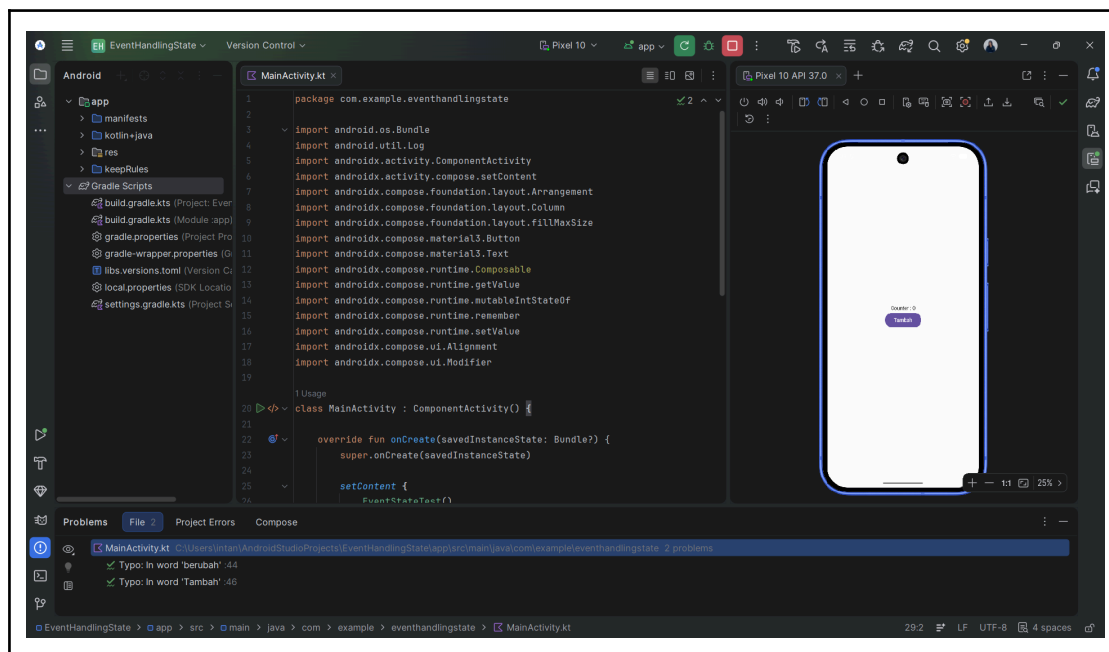
```
Log.d("TEST", "Counter berubah : $count")
```

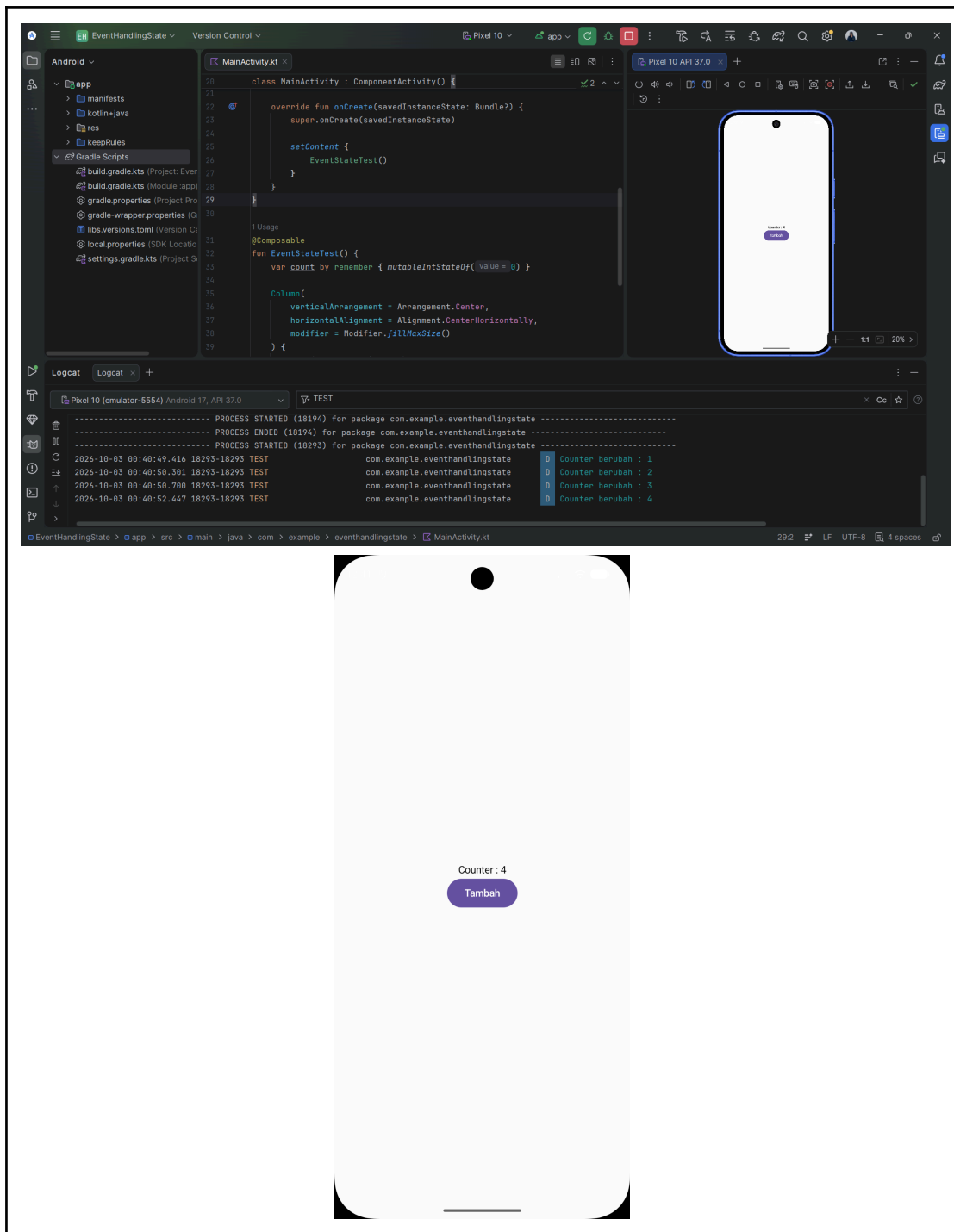
3. Tampilan counter di aplikasi tetap menunjukkan Counter : 0, walaupun nilai count di LogCat berubah.
4. Hal tersebut terjadi karena count masih menggunakan variabel biasa:

```
var count = 0
```

Variabel biasa dapat berubah nilainya, tetapi perubahan tersebut tidak otomatis memicu Compose untuk memperbarui UI. Agar tampilan ikut berubah, data perlu menggunakan state yang dapat diamati, seperti `mutableStateOf()`

5. Proses nomor 5 hingga 7





9. Di LogCat muncul nilai counter yang terus bertambah, ini berasal dari

```
Log.d("TEST", "Counter berubah : $count")
```

10. Nilai counter pada tampilan ikut bertambah setiap kali tombol diklik, bisa dilihat pada gambar berikut,



Berbeda dengan percobaan sebelumnya yang tampilannya tetap Counter : 0.

11. Perbedaannya terjadi karena sekarang count menggunakan state:

```
var count by remember { mutableStateOf(0) }
```

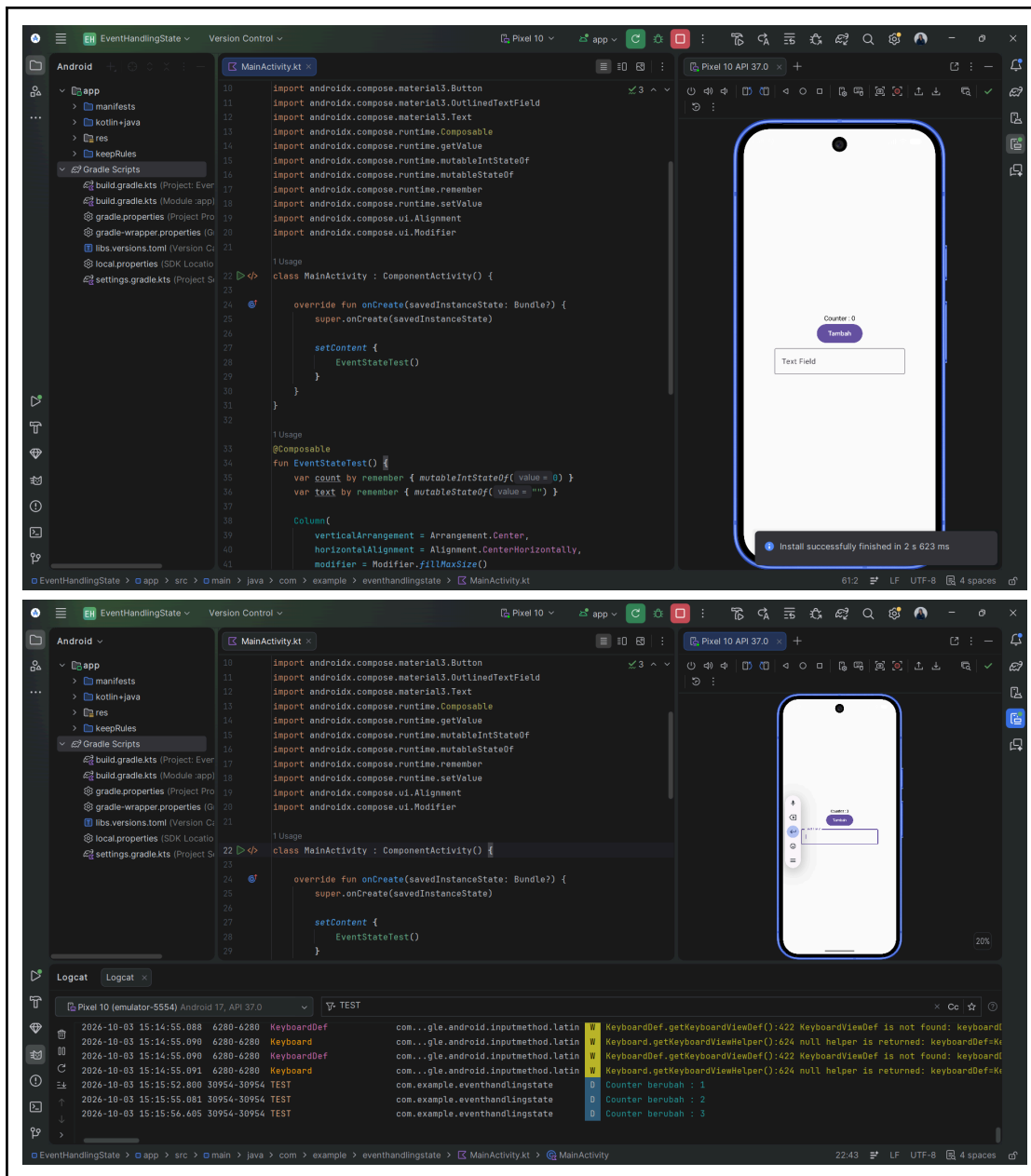
Perubahan pada state dapat diketahui oleh Compose. Ketika count berubah, Compose melakukan recomposition sehingga bagian UI yang menggunakan count diperbarui.

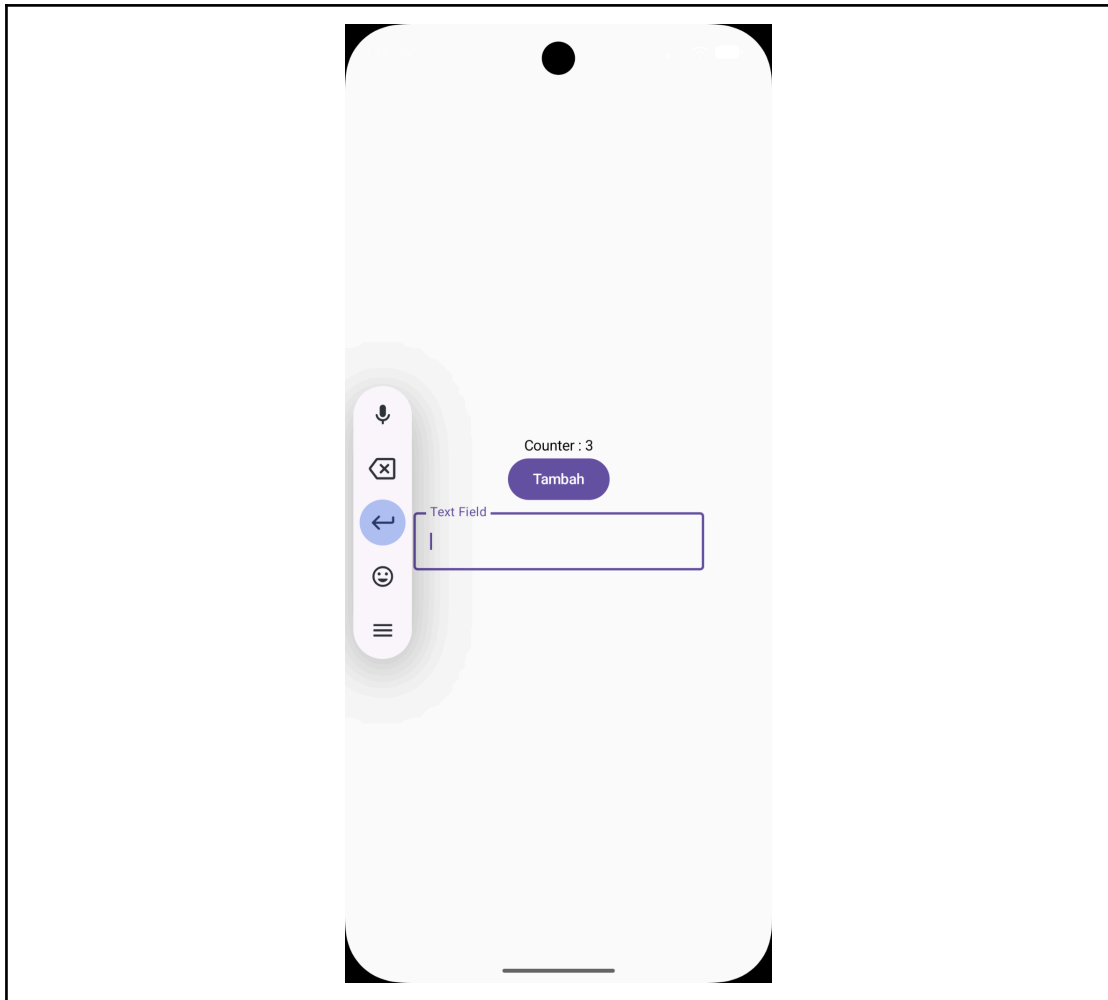
Pada percobaan sebelumnya, count hanya berupa variabel biasa:

```
var count = 0
```

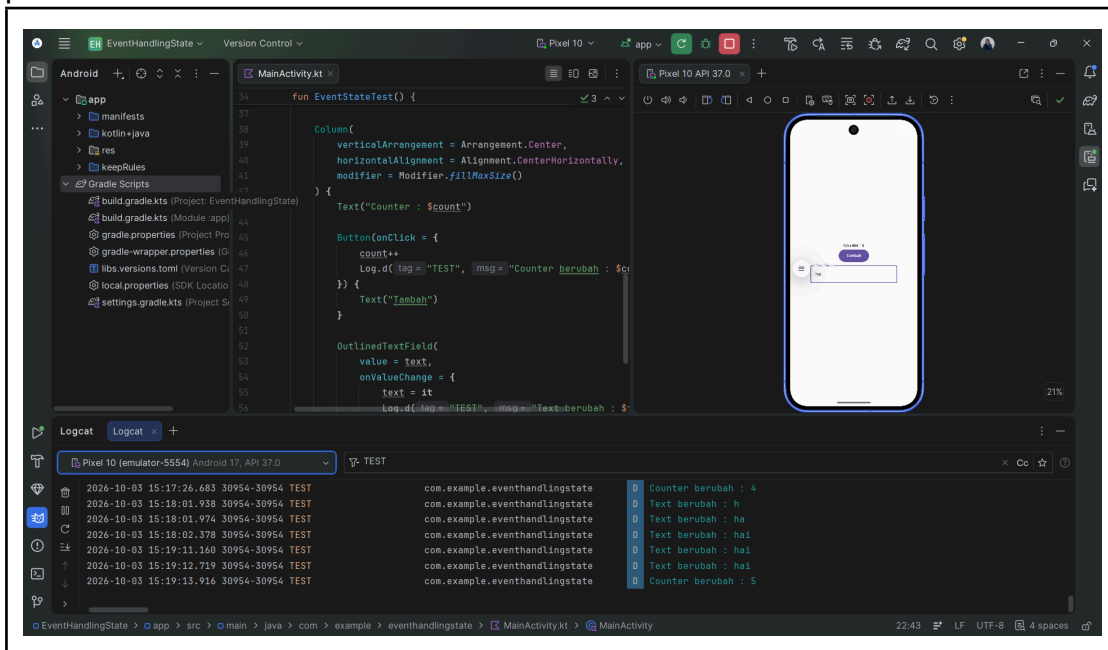
sehingga perubahan nilainya tidak otomatis membuat UI diperbarui.

12. Recomposition adalah proses pembaruan UI pada Jetpack Compose ketika state berubah. Compose akan memperbarui bagian tampilan yang menggunakan state tersebut.
13. Percobaan 13 hingga 14





15. Setiap kali kita mengetik huruf pada Text Field, LogCat akan menampilkan perubahan isi teks.



Hal tersebut berasal dari kode

```
onValueChange = {  
    text = it  
    Log.d("TEST", "Text berubah : $it")  
}
```

16. Setiap huruf yang diketik akan langsung muncul pada Text Field. Jadi isi Text Field akan terus mengikuti nilai yang diketik pengguna.

17. Hal tersebut terjadi karena nilai Text Field disimpan dalam state:

```
var text by remember { mutableStateOf("") }
```

Kemudian saat pengguna mengetik, `onValueChange` dijalankan dan nilai state `text` diperbarui:

```
text = it
```

Karena `text` merupakan state, perubahan nilainya menyebabkan Compose memperbarui tampilan melalui recomposition.

18. Terdapat dua state, yaitu

```
var count by remember { mutableStateOf(0) }  
var text by remember { mutableStateOf("") }
```

Jadi:

- `count` untuk menyimpan nilai counter.
- `text` untuk menyimpan teks yang dimasukkan ke Text Field.

19. Terdapat dua event, yaitu

a. `onClick` pada tombol Tambah.

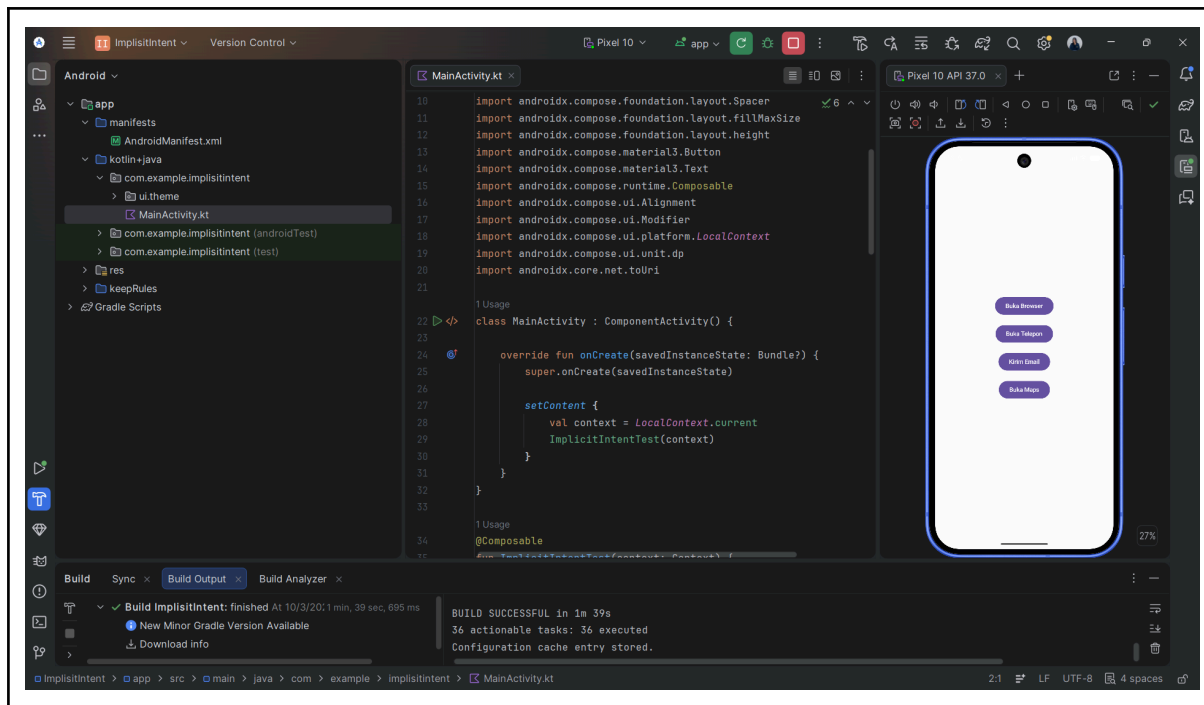
```
Button(onClick = {  
    count++  
}))
```

b. `onValueChange` pada `OutlinedTextField`.

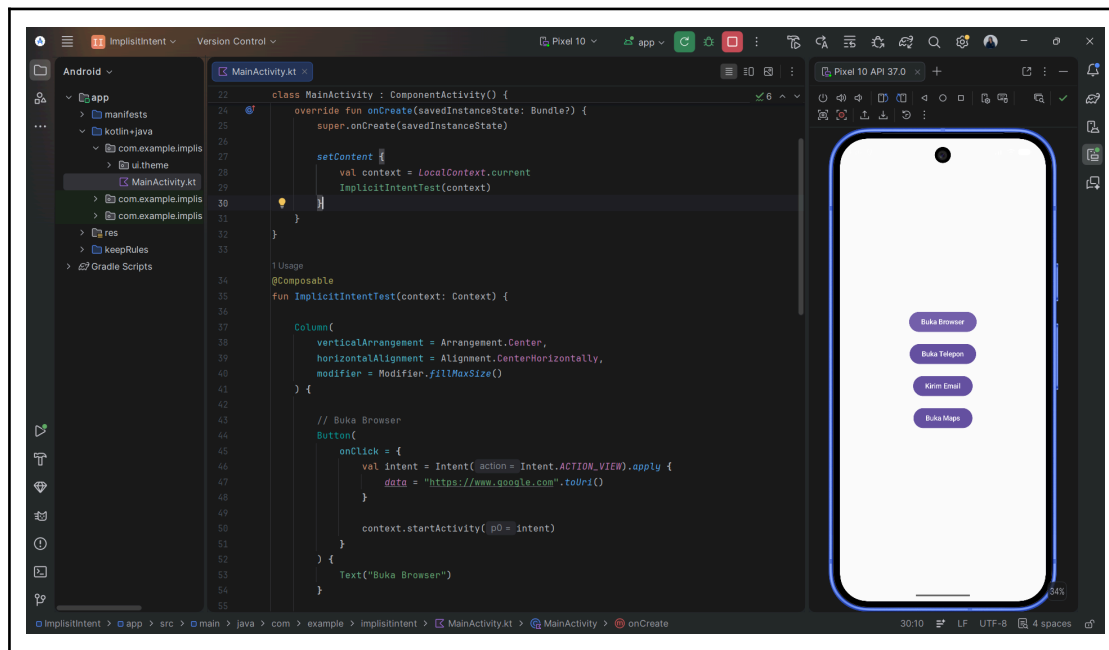
```
onValueChange = {  
    text = it  
}
```

20. `onClick` dijalankan ketika pengguna menekan tombol Tambah. Sedangkan, `onValueChange` dijalankan setiap kali isi Text Field berubah, misalnya saat pengguna mengetik atau menghapus karakter.

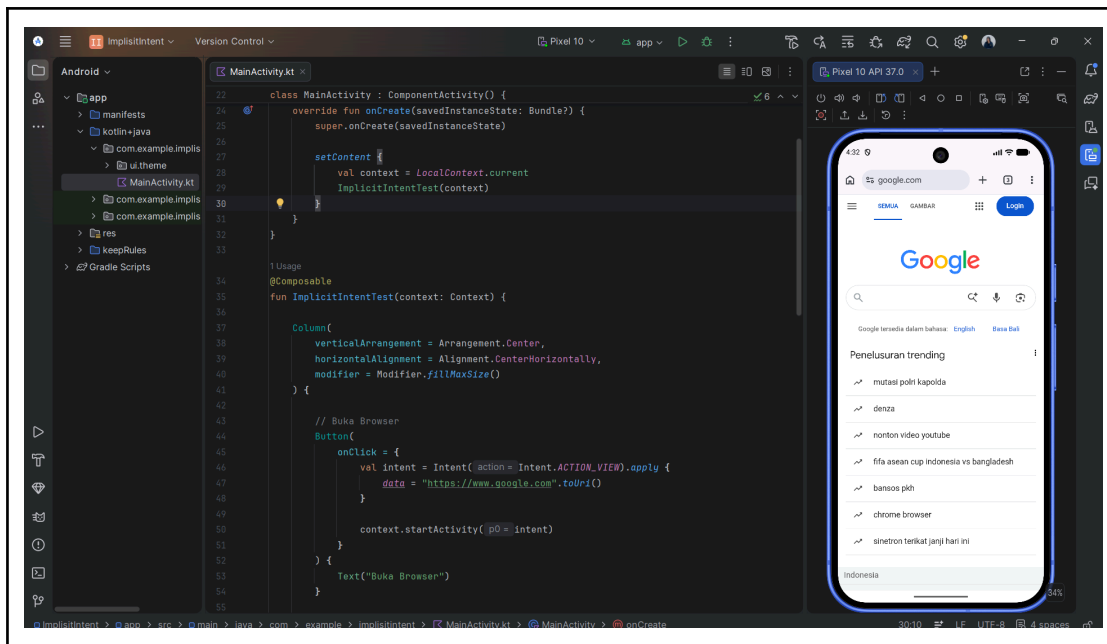
Percobaan 3 (Implisit Intent)



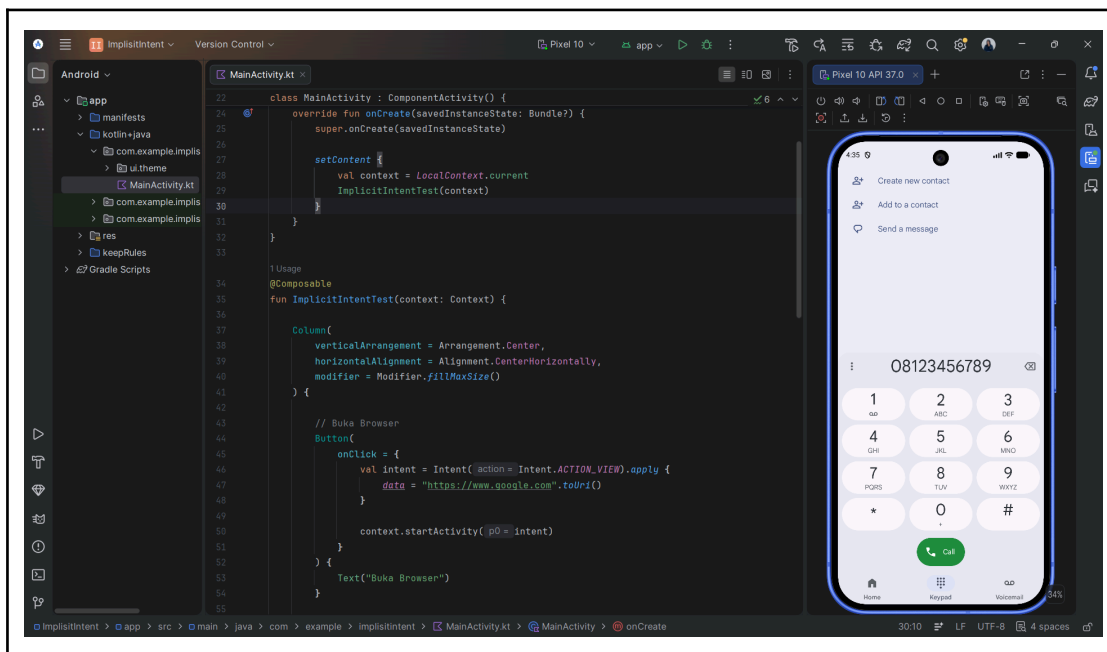
1. Setelah Run



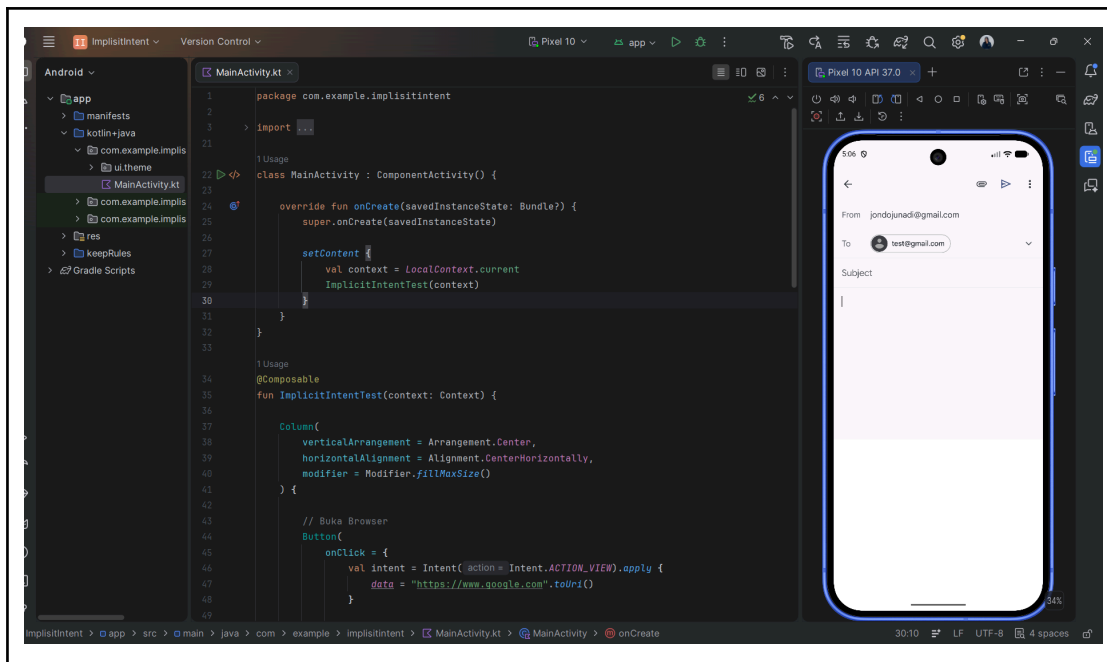
2. Buka Browser



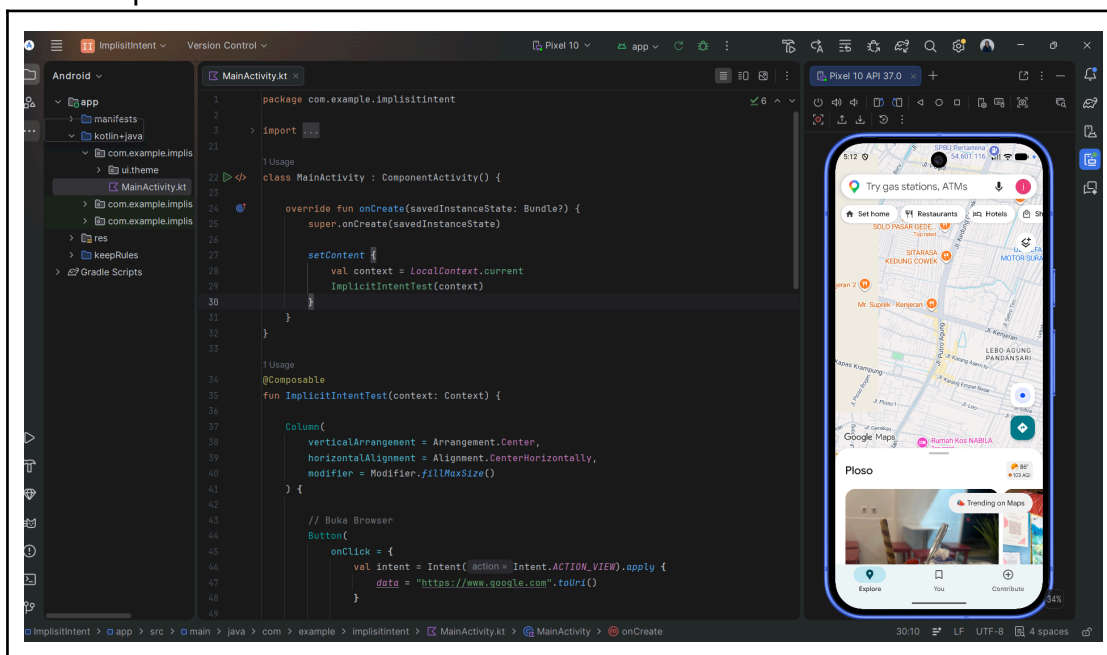
3. Buka Telepon



4. Kirim Email



5. Buka Maps



6. Hasil percobaan

- Buka Browser: aplikasi membuka browser dengan alamat <https://www.google.com>. Pada kode digunakan `Intent.ACTION_VIEW` dengan data berupa URL.
- Buka Telepon: aplikasi membuka aplikasi dial/telepon dengan nomor 08123456789 yang sudah terisi.
- Kirim Email: sistem menampilkan pilihan aplikasi email yang tersedia, lalu membuka aplikasi yang dipilih dengan alamat `test@gmail.com`.
- Buka Maps: sistem membuka aplikasi yang dapat menangani data lokasi geo:-7.250445,112.768845.

11. Perbedaan fungsi dari masing-masing action intent, antara lain:

- a. Intent.ACTION_VIEW digunakan untuk menampilkan atau membuka suatu data, misalnya URL website atau lokasi geo:. Action ini digunakan untuk membuka browser dan maps.
- b. Intent.ACTION_DIAL digunakan untuk membuka aplikasi telepon/dialer dengan nomor telepon yang sudah terisi, tetapi tidak langsung melakukan panggilan. Pada modul digunakan bersama data:

```
data = Uri.parse("tel:08123456789")
```

- c. Intent.ACTION_SENDTO digunakan untuk mengirim data ke aplikasi yang sesuai, pada percobaan ini digunakan untuk membuka aplikasi email dengan alamat tujuan tertentu.

16. Data berfungsi untuk memberikan informasi atau tujuan yang akan diproses oleh Intent. Fungsi masing-masing data pada inten

- a. menunjukkan alamat email tujuan

```
data = Uri.parse("mailto:test@gmail.com")
```

- b. menunjukkan alamat website yang akan dibuka.

```
data = Uri.parse("https://www.google.com")
```

- c. menunjukkan data lokasi yang akan dibuka.

```
data = Uri.parse("geo:-7.250445,112.768845")
```

17. Action menentukan aksi apa yang ingin dilakukan, sedangkan Data menentukan data atau tujuan yang akan digunakan untuk aksi tersebut. Android menggunakan kombinasi Action dan Data untuk menentukan aplikasi mana yang sesuai untuk menangani Intent.